

UNITAT 7 · SETMANES 10–12

Fils i concurrència

📅 Setmanes 10–12 · dates per confirmar </> Laboratori TD6 (resolt + repte IA)

~ 5 animacions

MANUAL M7 · TEORIA DE LA UNITAT

Fils i concurrència

Lectura bàsica de la unitat 7 (≈ 2 h). Les sortides són reals: programes del TD6 executats a la màquina del curs (28 nuclis x86-64).

CONTINGUT

§1	Fils	§4	Mutex, semàfors i variables de condició
§2	Condicions de cursa i atomicitat		
§3	Exclusió mútua: del programari al maquinari	§5	Interbloquejos

§1 Fils

Un **fil** és una execució seqüencial dins d'un procés. Els fils d'un procés comparteixen tot l'espai d'adreces —codi, dades globals, heap— i els descriptors; cadascun té la seva **pila**, els seus **registres** i la seva informació de planificació. No hi ha protecció de memòria entre ells: l'error d'un fil pot corrompre les dades de tots, i un senyal com SIGSEGV acaba tot el procés. A Linux, el nucli planifica cada fil com una tasca més (es creen amb `clone`, compartint memòria i descriptors).

L'API POSIX: `pthread_create(&t, attr, f, arg)` executa `f(arg)` en un fil nou; `pthread_exit` (o tornar de `f`) l'acaba; `pthread_join` n'espera el final, en recull el valor de retorn i n'allibera els recursos (o `pthread_detach`, si ningú no l'esperarà). Aquestes funcions retornen 0 o un número d'error, no -1. Compila amb `-pthread`. Crear un fil és molt més barat que crear un procés perquè no es copien taules de pàgines:

Sortida real

```
$ ./create_cost 2000
2000 threads:      41.2 us per pthread_create + pthread_join
2000 processes:   248.8 us per fork + waitpid
a process costs 6.0 times a thread
```

Sortida real

```
$ ./threads
thread 57632 of process 357789: shared[0x650ff476602c] = 1, local[0x7626e31fee74] = 1
thread 79520 of process 357789: shared[0x650ff476602c] = 4, local[0x7626e19fbe74] = 1
thread 19968 of process 357789: shared[0x650ff476602c] = 5, local[0x7626dbffee74] = 1
thread 64928 of process 357789: shared[0x650ff476602c] = 2, local[0x7626e29fde74] = 1
thread 72224 of process 357789: shared[0x650ff476602c] = 3, local[0x7626e21fce74] = 1
thread 86816 of process 357789: shared[0x650ff476602c] = 6, local[0x7626e11fae74] = 1
thread 94112 of process 357789: shared[0x650ff476602c] = 7, local[0x7626e09f9e74] = 1
thread 27264 of process 357789: shared[0x650ff476602c] = 8, local[0x7626db7fde74] = 1
process 357789: all 8 threads finished, shared = 8
```

Practica-ho → A7.4 fils contra processos · TD6 · 1

§2 Condicions de cursa i atomaticitat

Una línia de C com `x = x + 2` són diversos passos del processador: llegir `x` a un registre, sumar i escriure. Si dos fils executen línies així sobre la mateixa variable, el planificador pot intercalar-ne els passos de qualsevol manera, i en alguns ordres una escriptura **sobreescriu** l'altra: això és una **condició de cursa**. Per a `x = x+2 || x = x*3` amb `x = 1` hi ha 20 intercalats possibles; només els dos seqüencials donen 9 o 5, la resta perden una actualització. A la pràctica els ordres «dolents» són poc freqüents, i això fa que aquests errors siguin difícils de reproduir: al TD6, vuit fils que venen 100.000 entrades sense protecció en venen uns quants milers de més.

Una operació és **atòmica** si s'executa sencera o gens, sense que cap altre fil en vegi un estat intermedi. Des de C11, dos accessos a la mateixa variable des de fils diferents, sense sincronització i amb almenys una escriptura, són una *cursa de dades* i el comportament és indefinit; `volatile` no ho arregla. Per a operacions atòmiques, fes servir `<stdatomic.h>` o `__atomic_*`.

Sortida real

```
$ ./tickets_race 100000 | tail -3
[5] I sold 14026 tickets
[7] I sold 14753 tickets
103195 tickets sold out of 100000 (tickets = -7)
```

Practica-ho → A7.1 condició de cursa · TD6 · 2

§3 Exclusió mútua: del programari al maquinari

Una **secció crítica** és un fragment que només pot executar un fil cada vegada (**exclusió mútua**). El problema de «massa llet» mostra com és de difícil aconseguir-ho només amb lectures i

escriptures: mirar i comprar deixa una finestra; una nota compartida també (mirar-la i posar-la són dos passos); una nota per fil evita comprar dues vegades però permet que no compri ningú. L'**algorisme de Peterson** (nota pròpia + cedir el torn + esperar mentre l'altre té nota i és el seu torn) és correcte si la memòria és *seqüencialment consistent*.

Els processadors reals no ho són. A x86, cada nucli deixa les seves escriptures en una **memòria intermèdia** i pot llegir la memòria abans que siguin visibles per als altres (el model TSO); ARM reordena encara més. Així, els dos fils de Peterson poden llegir la nota de l'altre com a 0 i entrar junts. Mesurat a la màquina del curs, amb barrera de memòria (`mfence`) i sense:

Sortida real

```
$ for i in 1 2 3 4 5; do ./peterson; done; for i in 1 2 3 4 5; do ./peterson_fence; done
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
```

Per això l'exclusió mútua es construeix amb **instruccions atòmiques del maquinari**: test-and-set, compare-and-swap (`lock cmpxchg` a x86), fetch-and-add. Un pany que espera en un bucle (*spinlock*) crema CPU i pot patir *inversió de prioritat*; la solució de Linux és el **futex**: si el pany és lliure es pren amb una instrucció atòmica a l'espai d'usuari, i només si cal esperar es crida el nucli per dormir. Sense contenció, un mutex pràcticament no fa crides al sistema:

Sortida real

```
$ strace -f -c -e trace=futex ./lock_cost 1 100000; strace -f -c -e trace=futex
./lock_cost 8 100000
1 thread(s) x 100000 lock/unlock: counter = 100000, 48.4 ns per lock/unlock
0.00 0.000000 0 1 futex
8 thread(s) x 100000 lock/unlock: counter = 800000, 85.5 ns per lock/unlock
100.00 0.223730 27 8013 3116 futex
```

Practica-ho → A7.5 el problema de la llet · TD6 · 5

§4 Mutex, semàfors i variables de condició

Un **mutex** (`pthread_mutex_lock` / `unlock`) protegeix una secció crítica: qui troba el mutex agafat dorm fins que s'allibera. Un **semàfor** (Dijkstra) és un enter ≥ 0 amb dues operacions atòmiques:

`sem_wait` (P: espera que sigui positiu i resta 1) i `sem_post` (V: suma 1 i desperta un fil que espera). Inicialitzat a 1 és un pany; a 0, un avís entre fils; a k, un límit de k usuaris alhora. `sem_init` en crea un d'anònim (compartible entre processos amb `pshared = 1` en memòria compartida) i `sem_open` un amb nom (retorna `SEM_FAILED` si falla).

El **productor–consumidor** combina tots dos: una memòria intermèdia de `BUF` cel·les, un semàfor `empty` (cel·les lliures, comença a `BUF`), un `full` (cel·les ocupades, comença a 0) i un mutex per als índexs compartits. El productor espera si la memòria intermèdia és plena i el consumidor si és buida; no es perd ni es repeteix res:

Sortida real

```
$ ./prodcons_multi 4 100000
consumer 2: 23918 items
consumer 3: 26423 items
consumer 1: 23037 items
consumer 0: 26622 items
sum = 5000050000, expected 5000050000: OK
```

Per esperar una condició qualsevol es fan servir **variables de condició**: amb el mutex agafat, `while (!condition) pthread_cond_wait(&c, &m);`. `cond_wait` allibera el mutex i dorm de manera atòmica, i el torna a agafar en despertar; qui canvia la condició crida `pthread_cond_signal` o `_broadcast`. Es fa servir `while` perquè hi pot haver despertars espuris.

Practica-ho → A7.2 productor–consumidor · TD6 · 3

§5 Interbloquejos

Un **interbloqueig** és un conjunt de fils que s'esperen els uns als altres per sempre. Només apareix si es compleixen **les quatre condicions** (Coffman): exclusió mútua, retenir recursos mentre s'esperen altres, no expropiació i espera circular. Com que depèn de l'ordre d'execució, pot no aparèixer en milers de proves. El sopar dels filòsofs ho mostra: si cadascun agafa primer el bastonet de la dreta i després el de l'esquerra, tots poden acabar amb un a la mà esperant el del veí. N'hi ha prou de trencar una condició: un **ordre global** d'adquisició (la correcció del TD6), limitar quants competeixen (un «cambrer»), alliberar i tornar-ho a provar (`trylock`), o detectar cicles al graf d'assignació i recuperar-se. Al nucli Linux, *lockdep* comprova que els panys sempre es prenen en el mateix ordre.

Sortida real

```

$ timeout 3 ./philosophers_deadlock 3 | tail -8; echo "exit status: ${PIPESTATUS[0]}"
(124 = still blocked after 3 s)
7: hungry
2: hungry
3: hungry
1: hungry
0: hungry
6: hungry
4: hungry
5: hungry
exit status: 124 (124 = still blocked after 3 s)

```

Sortida real

```

$ ./philosophers 3 | tail -3
7: full
7: done after 3 meals
dinner finished: 8 philosophers x 3 meals

```

Practica-ho → A7.3 els filòsofs · TD6 · 4